# CSC311: Introduction to Machine Learning

## Project Report

Submitted by

Roy Gal, William Gao, Zehao Peng, Steven Qiao

University of Toronto
Fall 2025

# 1 Executive Summary

We explored random forest classifiers, naive Bayes classifiers, and multinomial logistic regression models. Our best-performing model was a random forest classifier, with a projected accuracy of 76.4%. The random forest's ability to model complex, likely nonlinear relationships between a mix of categorial and text features makes it suitable for this task.

# 2 Data Exploration

## 2.1 Summary of dataset

Each of 275 students contributed one data point for each class label (ChatGPT, Claude, Gemini), resulting in a perfectly balanced 3-class dataset. The features include:

**4 numerical features**: likelihood of academic use, frequency of suboptimal responses, frequency of references, frequency of user verification.

**2 categorical features**: most suitable tasks, least suitable tasks.

**3 text features**: most suitable tasks, least suitable task with explanation, method of user verification.

The distribution of each of numerical feature is shown in Figure 1. As expected, the mean and median are generally close to the neutral answer 3, although the mode is 4 for the first and last questions.

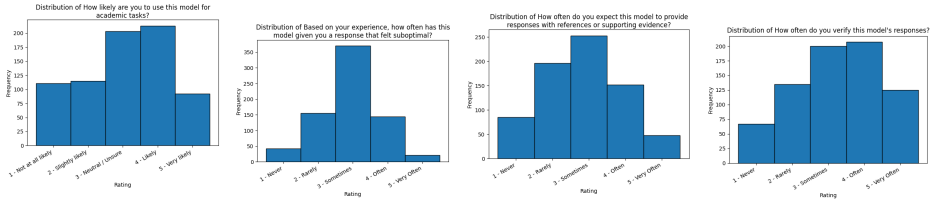



**Figure 1:** Distributions of numerical features.

## 2.2 Data issues

- The dataset is missing at least 118 values, e.g. student_id = 3. Missing numerical features were imputed as 3 (neutral); see `build_features` in `preprocessing.py`.
- Two data points (student_id = 2 and 15) contained a nonsensical feature "#NAME?". These were removed prior to training to prevent the model from learning from it.

## 2.3 Preprocessing

- To preserve ordinal information, Likert-scale features were converted to numerical by extracting the first character using regex pattern matching; see `extract_rating`

in `preprocessing.py`.
- Multi-select survey responses were converted into binary indicator features using MultiLabelBinarizer; see `build_features` in `preprocessing.py`.
- For logistic regression, the text data in the dataset was preprocessed by applying TF-IDF vectorization to each free-text column, converting the raw text into numerical features that capture the importance of each word relative to the whole; see `preprocessing_tfidf.py`.
- For the random forest classifier, a bag of words representation was implemented in `random_forest.ipynb`.

### 2.4 Data Splitting

- data-split.py groups the cleaned dataset by student_id and randomly splits 85/15 for training/test. The file training.csv was used for training and hyperparameter tuning; test.csv was only used to measure final model performance.

## 3 Methodology

### 3.1 Logistic Regression

Logistic (softmax) regression is a stable and efficient classifier for simple datasets such as this. It readily affords further techniques such as feature mapping to capture nonlinear relationships and regularization to prevent overfitting our small training set. We tuned a logistic regression model both including and excluding text features represented by TF-IDF, but observed that including TF-IDF features improved validation performance at the cost of higher variance between folds.

**Optimization:** For training, we used the built-in optimizers in the `LogisticRegression` class from scikit-learn. We chose the `lbfgs` solver for L2 regularization and `saga` solver for L1 regularization. We set `max_iter=5000` to ensure convergence when tuning hyperparameters, but used `max_iter=1000` for final testing to meet the computational requirements.

**Hyperparameters:** The optimal combination of hyperparameters was selected by a grid search; see `log_reg_tuning` and `log_reg_tfidf_tuning`. The search range for each hyperparameter is justified below.

- `max_features`: the maximum number of tokens considered in the TF-IDF representation of text features. We searched over the range $\{100, 300, 500, 1000\}$, and validation results (see Figure 2) showed that accuracy, precision, recall, and F1 score are all maximized at `max_features` $= 300$, providing evidence that this range is reasonable for our dataset.
- `n-gram range`. Allows the TF-IDF representation to consider short phrase combinations, rather than just individual words. We tried introducing bigrams (two word combinations), searching over the range `n-gram range` $\in \{(1,1),(1,2)\}$. However

we noticed immediate overfitting as $n$ increased as the mean training accuracy was 0.993 and the mean validation accuracy was 0.674 for $(1, 2)$, as opposed to mean training accuracy 0.854 and mean validation accuracy 0.688 for $(1, 1)$.

- **Regularization strength:** $C \in \{0.1, 0.5, 1, 5, 10\}$, controlling the inverse of regularization. From Figure 2, we observe the trend of slightly underfitting when $C$ is too small, thus regularization is too strong, and overfitting when $C$ is too large.
- **Penalty:** We experimented with both L1 and L2 regularization. L2 regularization encourages smaller coefficients and generally yields stable models, while L1 can produce sparse coefficients, potentially useful if some features are irrelevant. We could have also tried elasticnet which combines L1 and L2, but we found that L2 significantly outperformed L1.
- **Solver:** We used `lbfgs` for L2 regularization and `saga` for L1 regularization, as recommended by the `scikit-learn` documentation [2].

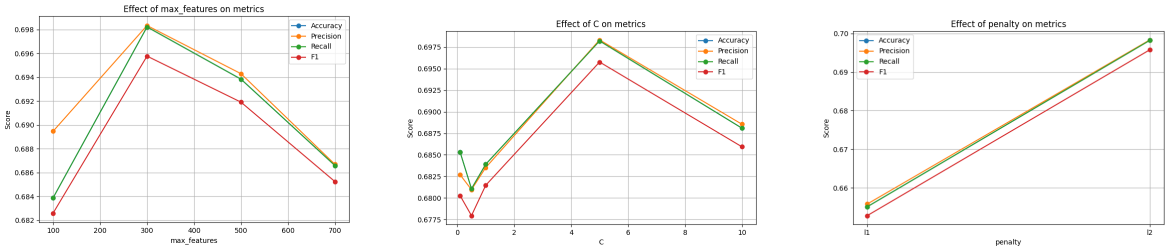



**Figure 2:** Logistic regression validation results as hyperparameters vary.

## 3.2 Naive Bayes Classifier

Naive Bayes is well-suited for our dataset because most survey features are discrete, low-cardinality variables such as Likert-scale responses and binary indicators. The naive Bayes assumption may not be perfectly true, for example the numerical features regarding likelihood of academic use and frequency of suboptimal responses are plausibly correlated. However, it seems reasonable to assume that features are approximately conditionally independent given class, hence a naive Bayes classifier can be very effective.

**Optimization:** Naive Bayes does not require iterative optimization. Parameters have closed-form maximum-likelihood estimates derived from empirical counts (categorical features) or means/variances (numerical features).

**Hyperparameters:** We tuned the Laplace smoothing coefficient $\alpha$, which addresses the zero-probability problem in Bayesian classifiers by adding a small constant $\alpha$ to each feature count. The search range for $\alpha$, namely $\alpha \in \{0.01, 0.1, 0.5, 1.0, 2.0, 5.0, 10.0\}$, was chosen following the recommendation of Manning, Raghavan, and Schütze [1]. The effect of $\alpha$ on the evaluation metrics is shown in Figure 3.

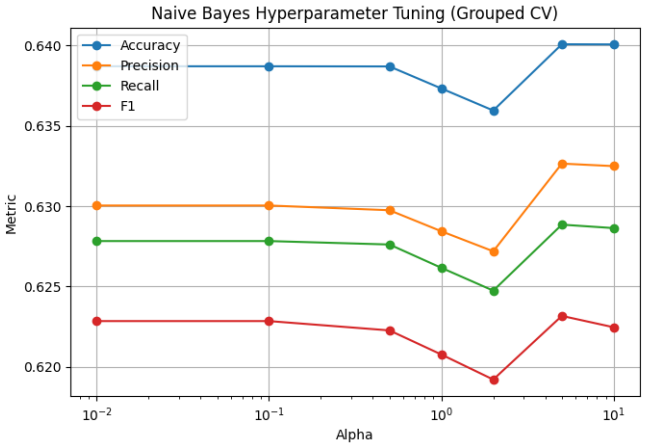



**Figure 3:** Naive Bayes validation results as the Laplace smoothing coefficient varies.

## 3.3 Random Forest Classifier

Decision trees are suitable for mixed feature types and capturing relationships between numerical features and binary categorical features, which form an important part of our dataset. The numerical features exhibit relatively concentrated distributions, with the most concentration near rating 3 to 4. This means sampled values are more predictable, and Gini impurity from numerical feature splits can be quite informative. We can extend this to random forests, using ensemble averaging to avoid overfitting.

**Optimization:** We used the Gini impurity criterion for sklearn's `RandomForestClassifier` class, which measures how mixed the class labels are. Since the hyperparameters do not interact, it is not necessary to perform a grid search; it suffices to tune each hyperparameter separately.

**Hyperparameters:** See `random_forest.ipynb` for implementation details.

- `vocab_size`: the number of words considered in the bag of words representation of text features. The search range

$$\{1, 10, 50, 100, 500, 1000, 1500, 2000, 2500, 3000, 3500, 4000, 4500, 5000, 5687\}$$

  ranges from a single word to the total number of distinct words in the dataset, 5687.
- `n_estimators`: the number of trees in the random forest. `range`$(1, 1000, 10)$ covers a very broad range, and the validation results (see Figure 4) shows a visible initial learning curve and eventual smoothing, justifying this search range.
- `max_depth`: the maximum depth of a tree in the random forest. Again, the search range was `range`$(1, 1000, 10)$, and is justified by the initially steep learning rate and almost immediate smoothing.
- `max_features`: the maximum number of features considered at each split in a tree. We searched among 'sqrt', 'log2', 0.2, 0.5, 1.0, but the validation accuracy for 'sqrt' was by far the highest.

- `min_samples`: the minimum number of samples required to be a leaf node. In the search range $\{1, 2, 5, 10, 20, 50\}$, `min_samples` $= 1$ attained the highest validation accuracy by a large margin.

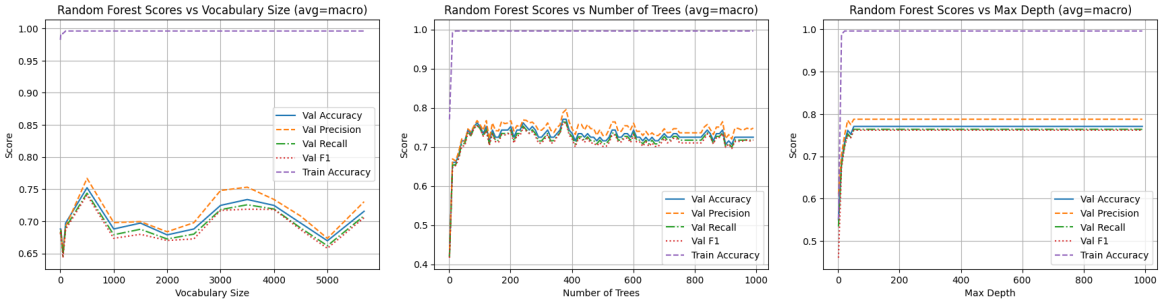



**Figure 4:** Random forest classifier validation results as the key hyperparameters vary.

## 3.4 Validation

We employed grouped 5-fold cross-validation using `GroupKFold`, grouping by `student_id` to avoid data leakage across folds (see `logistic_regression_tfidf.py`) as well as hold-out validation with a 70/15/15 split for training/validation/test.

## 3.5 Evaluation

We evaluated our models using accuracy, precision, recall, and F1. The final decision was heavily based on accuracy and F1, which combines precision and recall.

# 4 Results

## 4.1 Logistic Regression

Table 1 summarizes the best combination of hyperparameters for logistic regression. Table 3 contains the resulting test performance metrics, overall and on each class.

| Model | Penalty | Solver | C | Max Features | N-grams |
|---|---|---|---|---|---|
| TF-IDF | L2 | saga | 5 | 300 | (1,1) |
| No Text | L1 | saga | 0.1 | N/A | N/A |

**Table 1:** Logistic regression hyperparameters with strongest 5-fold CV performance.

**Error Analysis:** As evidenced by the confusion matrix (see Figure 5) and the evaluation metrics (see Table 3), the model achieved high accuracy (0.818) and F1 (0.847) for ChatGPT, but often confused Claude (F1: 0.564) and Gemini (F1: 0.554). The logistic regression classifier exhibited consistency between precision and recall, which suggests it is well-balanced in handling false positives and false negatives across classes, but frequently confuses Claude and Gemini.

## 4.2 Naive Bayes Classifier

The optimal Laplace smoothing coefficient was $\alpha = 5.0$. The test performance of this model is summarized in Table 3.

**Error Analysis:** From the confusion matrix, we see that the naive Bayes classifier identified ChatGPT with high recall (0.875), but lower precision (0.648). It suffered with low recall (0.382) for Gemini, which it confused the most with Claude but also significantly with ChatGPT. It demonstrated balanced but mediocre precision (0.529) and recall (0.546) for Claude. This suggests that the naive Bayes classifier suffer from many false positives for ChatGPT and many false negatives for Gemini.

## 4.3 Random Forest Classifier

Table 2 shows the best hyperparameters for random forests, each tuned independently.

| **Hyperparameter** | `vocab_size` | `n_estimators` | `max_depth` |
|---|---|---|---|
| **Value** | 500 | 371 | 51 |
| **Validation Score** | 0.752 | 0.771 | 0.771 |

**Table 2:** Logistic regression hyperparameters with strongest 5-fold CV performance.

**Error Analysis:** The random forest classifier is the overall strongest according to every metric, although according to a per-class analysis it suffers from a low precision for ChatGPT predictions (0.644) and an extremely low recall for Gemini (0.267). Similarly to the naive Bayes classifier, the random forest suffers from many false positives for ChatGPT and false negatives for Gemini.

## 4.4 Model Comparison

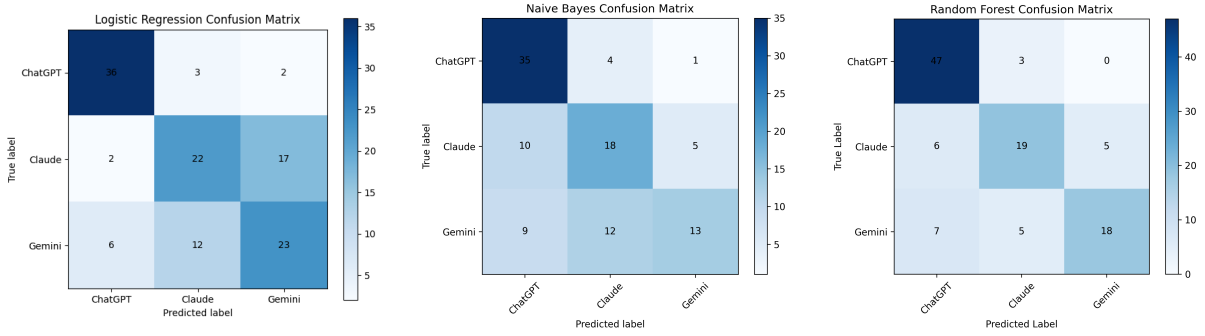



**Figure 5:** Confusion matrices for logistic regression, naive Bayes, and random forest models.

The model families from strongest to weakest performance were: random forest, logistic regression, naive Bayes. The estimated test performance of the random forest classifier is 76.4%. Since its performance on our test was very consistent (within 1% accuracy and F1) with the validation performance, we simply used the test accuracy for our estimate.

| **Macro** | **Accuracy** | **Precision** | **Recall** | **F1 Score** |
|---|---|---|---|---|
| Logistic Regression (with TF-IDF) | 0.659 | 0.653 | 0.659 | 0.655 |
| Naive Bayes | 0.617 | 0.621 | 0.601 | 0.591 |
| Random Forest | 0.764 | 0.757 | 0.724 | 0.733 |
| **ChatGPT** | | **Precision** | **Recall** | **F1 Score** |
| Logistic Regression | | 0.818 | 0.878 | 0.847 |
| Naive Bayes | | 0.648 | 0.875 | 0.745 |
| Random Forest | | 0.644 | 0.940 | 0.764 |
| **Claude** | | **Precision** | **Recall** | **F1 Score** |
| Logistic Regression | | 0.595 | 0.537 | 0.564 |
| Naive Bayes | | 0.529 | 0.546 | 0.537 |
| Random Forest | | 0.593 | 0.533 | 0.561 |
| **Gemini** | | **Precision** | **Recall** | **F1 Score** |
| Logistic Regression | | 0.548 | 0.561 | 0.554 |
| Naive Bayes | | 0.684 | 0.382 | 0.491 |
| Random Forest | | 0.800 | 0.267 | 0.400 |

**Table 3:** Evaluation metrics for the three model families on the test set.

# 5 Contributions and Learning

- Roy Gal: I was mainly responsible for implementing the logistic regression classifier. I learned how to use a TF-IDF representation for text features, even though it ultimately had a small impact on the model's accuracy.
- William Gao: My main contribution was formulating the project methodology and writing the proposal and final report. The most important lesson I learned is to use a variety of evaluation metrics when selecting a final model.
- Zehao Peng: I contributed to this project by implementing and optimizing the random forest classifier. I learned that when hyperparameters do not interact, it suffices to tune them individually rather than to perform a grid search.
- Steven Qiao: I contributed to this project by constructing the Naive Bayes model using the given training data. In the process of training and validating, I discovered that text features only marginally improved the accuracy of the model but computationally ensued a lot of overhead. I was able to make the optimal decision by considering the trade-off.

# References


[1] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008.

[2] scikit learn. sklearn.linear_model.logisticregression — scikit-learn 1.6.2 documentation. `https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html`, 2025.